

Functional and Non-Functional Requirements

Software Engineering Study Notes

Requirements engineering defines what a software system must do and the conditions under which it must operate. Two of the most important requirement categories are functional requirements and non-functional requirements.

1. Functional Requirements

Definition: Functional requirements describe the specific services, behaviors, operations, and features that the system must provide. They answer the question: "What should the system do?"

- Describe system behavior in response to inputs or events.
- Define business rules, calculations, data processing, and user actions.
- Usually map directly to use cases, user stories, API operations, or system functions.
- Can often be verified by checking whether a specific function produces the expected result.

Examples of Functional Requirements

- The system shall allow users to create an account using an email address and password.
- The system shall authenticate users before allowing access to private data.
- The shopping system shall calculate the total order price including taxes and discounts.
- The banking application shall allow a customer to transfer money between eligible accounts.
- The study assistant shall allow a user to create, list, update, complete, and delete study tasks.
- The system shall generate a monthly sales report from stored transaction data.

2. Non-Functional Requirements

Definition: Non-functional requirements describe quality attributes, constraints, and operational expectations of the system. They answer questions such as: "How well should the system work?", "How fast?", "How secure?", and "Under what constraints?"

- Define measurable quality characteristics of the software.
- Often apply across many or all system functions.
- Influence architecture, infrastructure, technology choices, and testing strategy.
- Should be written in measurable terms whenever possible.

Common Non-Functional Requirement Categories

Category	Meaning	Example
Performance	How quickly the system responds or processes work.	95% of requests shall complete within 2 seconds.
Scalability	Ability to handle increasing users, data, or workload.	The service shall support 10,000 concurrent users.
Availability	How often the system is operational.	The system shall provide 99.9% monthly uptime.
Reliability	Ability to operate correctly over time.	The system shall recover failed jobs without data loss.
Security	Protection of data, identities, and operations.	Passwords shall be stored using a strong salted hash.
Usability	Ease of learning and using the system.	A new user shall complete registration in under 3 minutes.

Maintainability	Ease of modifying, testing, and supporting the system.	Core modules shall have automated unit tests.
Portability	Ability to operate in different environments.	The application shall run on current Linux and Windows versions.
Compliance	Legal, regulatory, or standards constraints.	Personal data shall be processed according to applicable privacy rules.

3. Functional vs. Non-Functional Requirements

Aspect	Functional	Non-Functional
Main Question	What does the system do?	How well does the system operate?
Focus	Features and behavior	Quality attributes and constraints
Example	User can reset a forgotten password.	Password reset request returns within 2 seconds.
Testing	Functional, integration, acceptance tests	Performance, security, usability, reliability tests
Scope	Often tied to a specific feature	Often affects the whole system
Architecture Impact	May affect architecture	Frequently drives architecture strongly

4. How to Write Good Requirements

A strong requirement should be:

- Clear - it should have one understandable meaning.
- Specific - avoid vague words such as fast, easy, secure, or user-friendly without a measurable definition.
- Testable - a tester should be able to verify whether it has been satisfied.
- Feasible - it must be realistic within technical, budget, and schedule constraints.
- Consistent - it should not conflict with another requirement.
- Traceable - it should be connected to a business need, stakeholder goal, or source.

5. Good and Bad Non-Functional Examples

Weak Requirement	Improved Requirement
The system should be fast.	95% of API requests shall return within 2 seconds under a load of 500 concurrent users.
The application should be secure.	The application shall require multi-factor authentication for administrator accounts.
The interface should be easy to use.	At least 90% of first-time test users shall complete the checkout flow without assistance.
The system should be reliable.	The system shall maintain 99.9% monthly availability excluding scheduled maintenance.

6. Example: Online Library System

Functional requirements:

- Users shall search for books by title, author, or ISBN.
- Users shall reserve an available book.
- Librarians shall add, update, and remove book records.
- The system shall calculate overdue fines.

Non-functional requirements:

- Search results shall be displayed within 2 seconds for 95% of requests.
- The system shall support at least 2,000 concurrent users.
- Sensitive user data shall be encrypted in transit using TLS.
- The service shall achieve 99.9% monthly availability.

7. Key Takeaway

Functional requirements define the capabilities and behavior of the system, while **non-functional requirements** define the quality level, constraints, and operational characteristics of those capabilities. A successful software specification needs both.

Quick memory rule: Functional = WHAT the system does. Non-functional = HOW WELL the system does it.